Morse Code Converter

The Deep Dive

An exhaustive, beginner-safe walkthrough of every file,
every function, and every design decision in the project.

This document assumes no prior programming knowledge. Every technical term is defined at the point where it first appears. Everything stated here was read out of the source code or confirmed by running it; anything inferred is labelled as inferred.

Contents

1 What this project is

The Morse Code Converter is a small desktop application, written in Python, that turns ordinary text (`SOS`) into International Morse Code (`... --- ...`) and turns Morse code back into ordinary text. It does this live, as you type, in a window with two tabs. It can also play the resulting Morse code out loud as real beeps, while flashing each dot and dash on screen in time with the sound.

The whole project is three Python files and roughly 600 lines of code including comments. It has no third-party dependencies whatsoever: everything it does, it does with what ships inside Python itself.

Jargon: Morse code

A way of representing letters and numbers as sequences of two symbols: a short signal (a *dot*, written `.`) and a long signal (a *dash*, written `-`). It was designed in the 1830s for sending messages down a telegraph wire, where the only thing you could control was whether current was flowing or not. The letter S, for example, is three dots (`...`) and the letter O is three dashes (`---`), which is why the distress call SOS is written `... --- ...`. "International" Morse Code (also called ITU Morse) is the modern standardised version, as opposed to the older American Morse.

Jargon: GUI (Graphical User Interface)

The part of a program you see and click: windows, buttons, text boxes. The opposite is a *command-line interface* (CLI), where you type a command into a terminal and read text back. This project has both, but the GUI is the main event.

Jargon: Module

A single Python file containing code. One module can *import* another, which means "load that file and let me use the things defined inside it". Splitting a program into modules is how you keep unrelated concerns in separate places.

Why the project is shaped the way it is

The single organising decision is that the **conversion logic knows nothing about the interface**, and the **audio logic knows nothing about the interface either**. Two of the three files can be imported and used with no window ever opening. Only the third file, `gui.py`, knows that a screen exists. This is what makes the logic testable and the audio reusable, and it is the first thing to point at when explaining the project.

2 The shape of the repository

```
morse-code-converter
├── morse_converter.py     the alphabet and the two conversions (no UI, no audio)
├── morse_audio.py         timing, tone generation, playback backends (no UI)
├── gui.py                 the tkinter window; imports the other two
├── requirements.txt       declares that there are no dependencies
├── README.md              project documentation
├── LICENSE                PolyForm Strict 1.0.0
├── .gitignore             files git should not track
└── docs/
    ├── screenshots/
    │   └── app.png        a screenshot of the running window
    └── explainers/
        ├── deep-dive.tex  this document
        └── brief.tex      the shorter companion
```

Figure 1: Every tracked file in the project. Three of them contain code.

The file-organisation rationale is visible directly in the import statements: the arrows only ever point one way.

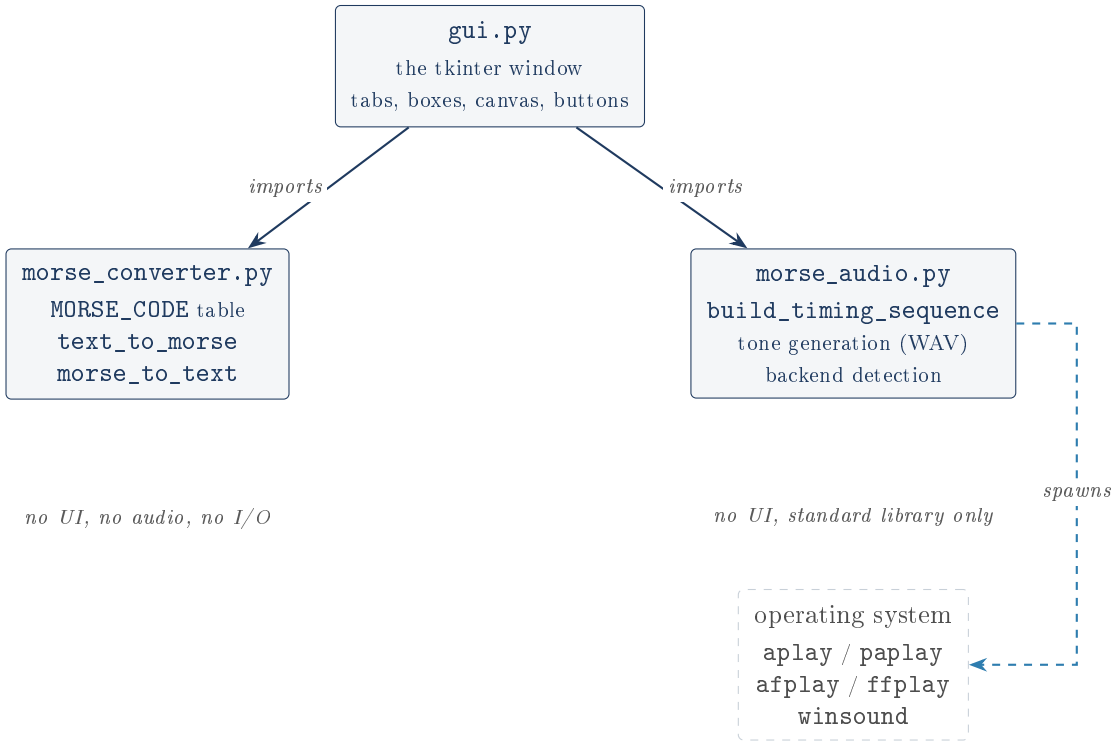

Figure 2: Module dependency graph. Nothing points upward: neither `morse_converter.py` nor `morse_audio.py` imports `gui.py`, and they do not import each other in normal use.

One import that crosses the line, in a corner

`morse_audio.py` does import `morse_converter.py`, but only *inside* its `if __name__ == "__main__":` block, which runs solely when you execute that file directly as a smoke test. So the clean separation holds for every real code path, and the exception is deliberate and contained. It is worth knowing about rather than claiming the modules are perfectly independent.

Jargon: `if __name__ == "__main__":`

A standard Python idiom. Python sets the variable `__name__` to the string `"__main__"` when a file is run directly (`python3 gui.py`), and to the file's own name when the file is merely imported by another file. So code guarded by this line runs only in the first case. It is how one file can be both an importable library and a runnable script.

3 `morse_converter.py`: the alphabet and the two conversions

This is the smallest and most important file: 62 lines, no imports at all.

3.1 `MORSE_CODE`: the lookup table

Jargon: Dictionary (in Python: `dict`)

A container that stores pairs, mapping a *key* to a *value*, and lets you retrieve the value instantly by naming the key. Think of a physical dictionary: you look up "aardvark" (the key) and get its definition (the value). Crucially, the lookup does not get slower as the dictionary grows, because it uses a *hash table* internally: it computes a number from the key and jumps straight to the right slot rather than scanning every entry.

The heart of the program is a hardcoded dictionary mapping each supported character to its Morse pattern:

```
MORSE_CODE = {
    "A": ".-", "B": "-...", "C": "-.-.", "D": "-..", "E": ".",
    "F": "..-.", "G": "--.", "H": "....", "I": "..", "J": ".---",
    "K": "-.-", "L": ".-..", "M": "--", "N": "-.", "O": "---",
    "P": ".--.", "Q": "--.-", "R": ".-.", "S": "...", "T": "-",
    "U": "..-", "V": "...-", "W": ".--", "X": "-..-", "Y": "-.--",
    "Z": "--..",
    "0": "-----", "1": ".----", "2": "..---", "3": "...--", "4": "....-",
    "5": ".....", "6": "-....", "7": "--...", "8": "---..", "9": "----.",
    " ": "/",
    "!": "-.-.--", "?": "..--..", ".": ".-.-.-", ",": "--..--",
    "'": ".----.", '"': ".-..-.", "/": "-..-.", "(": "-.--.",
    ")": "-.--.-", "&": ".-...", ":": "---...", ";": "-.-.-.",
    "=": "-...-", "+": ".-.-.", "-": "-....-", "_": "..--.-",
    "$": "...-..-", "@": ".--.-.", "¿": "..-.-", "¡": "--...-",
}
```

Listing 1: The complete table, reproduced exactly as it appears in `morse_converter.py`. This is the whole of the project's domain knowledge: every other line of code exists to move data through this dictionary.

The table holds **57 entries** (confirmed by loading the module and counting): 26 letters, 10 digits, the space character, and 20 punctuation marks including the inverted Spanish ¿ and ¡.

Two rows repay a second look. `"O": "---"` is three dashes, the longest of the plain letters alongside `"0": "-----"`, which is five; and the hyphen character itself is an entry, `"-": "-....-"`,

so a dash can be spelled in dashes. Both are worth noticing because they are the rows that a document about Morse code is most tempted to quietly skip.

The most consequential row is `" ": "/"`. The space character is treated as just another character in the alphabet, whose Morse "pattern" is a forward slash. This one decision quietly does a lot of work, as we will see in Section 3.6.

3.2 TEXT_CODE: the table, inverted

```
1 TEXT_CODE = {value: key for key, value in MORSE_CODE.items()}
```

Jargon: Dictionary comprehension

A compact way of building a dictionary from another collection in one line. Read `{value: key for key, value in MORSE_CODE.items()}` as: "go through every key/value pair in `MORSE_CODE`, and build a new dictionary in which each old value becomes the new key and each old key becomes the new value". The result is the same table read backwards: `".-"` now maps to `"A"`.

This is a neat trick, and it means there is exactly one place to edit if a mapping is wrong. But it carries a silent hazard.

The inversion hazard, and why this project escapes it

Dictionary keys must be unique. If two different characters had the *same* Morse pattern, inverting the table would silently discard one of them: no error, no warning, just a decoder that is quietly wrong for one character forever. Nothing in the code checks for this.
I checked it directly rather than assuming. Loading the module and counting gives **57 entries in `MORSE_CODE` and 57 in `TEXT_CODE`**, with zero duplicate values. So the inversion is lossless today, and the code is correct. The hazard is that it is correct *by luck of the table's contents*, not by any guard, and a future edit that introduced a duplicate would not be caught.

3.3 text_to_morse: encoding

```
1 def text_to_morse(text: str) -> str:
2     """Convert a string of text to Morse code, characters separated by spaces."""
3     codes = []
4     for char in text.upper():
5         if char not in MORSE_CODE:
6             raise ValueError(f"No Morse mapping for character: {char!r}")
7         codes.append(MORSE_CODE[char])
8     return " ".join(codes)
```

Line by line:

- `text: str -> str` is a *type hint*: a note to the reader (and to automated checkers) that this function takes a string and returns a string. Python does not enforce it at runtime.
- `text.upper()` converts the whole input to capitals first. This is the entire implementation of the "case-insensitive" feature: the table only contains capitals, so lowercase input is normalised before it is ever looked up.
- The `if char not in MORSE_CODE` check is the deliberate part. Without it, `MORSE_CODE[char]` on an unknown character would raise a `KeyError`, a low-level error naming only the character with no context. Instead the function raises a `ValueError` carrying a human-readable message. The `!r` inside the f-string means "show this using `repr`", so an invisible character prints as `'\n'` rather than as an actual blank line.

- `" ".join(codes)` glues the per-character patterns together with a single space between each.

Jargon: Raising an exception

When a function cannot do its job, it can *raise* an exception: it stops immediately and hands an error object back up to whoever called it. The caller can *catch* it (with `try`/`except`) and react, or let it propagate and crash the program. `ValueError` conventionally means "you gave me a value I cannot work with".

So `text_to_morse("SOS")` walks S, O, S and produces `... --- ...`: the canonical Morse message, three dots, three dashes, three dots. Add a second word and the space rule shows itself: `text_to_morse("SOS SOS")` gives `... --- ... / ... --- ...`. Note that the space became `/` through the ordinary table lookup, and then got a space on each side from the `join`, which is why every word boundary in the output is literally the three characters `" / "`.

3.4 morse_to_text: decoding

```
def morse_to_text(morse: str) -> str:
    words = morse.strip().split(" / ")
    decoded_words = []
    for word in words:
        letters = []
        for symbol in word.split():
            if symbol not in TEXT_CODE:
                raise ValueError(f"No text mapping for Morse symbol: {symbol!r}")
            letters.append(TEXT_CODE[symbol])
        decoded_words.append("".join(letters))
    return " ".join(decoded_words)
```

This is the mirror image, in two nested levels:

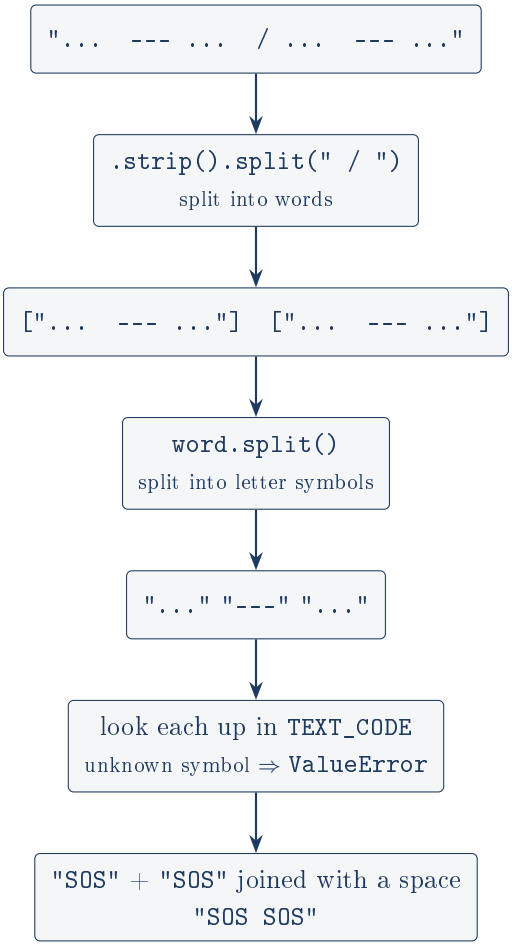

Figure 3: The decoding pipeline for `"SOS SOS"`: split on words, then on letters, then look up each symbol. Both words decode through the same three lookups, `"..."` → S, `"---"` → O, `"..."` → S.

`.strip()` removes leading and trailing whitespace so a pasted string with a stray newline still decodes. `word.split()` with no argument splits on any run of whitespace, so extra spaces between letters are tolerated.

3.5 The round-trip property, tested

The docstring claims that `morse_to_text(text_to_morse(s)) == s.upper()`. That is a strong claim, so I tested it rather than repeating it. Running the two functions over letters, digits, punctuation, mixed case, leading and trailing spaces, doubled internal spaces, an all-whitespace string, and a string containing a literal forward slash (`"ABC/DEF"`, where `/` is itself a table entry mapping to `-..-.`) gave a correct round trip in every case. The claim holds for the inputs the encoder accepts.

3.6 The redundant word-splitting layer

The outer `split(" / ")` does nothing the table was not already doing

`morse_to_text` handles word gaps twice. It splits the input on `" / "`, decodes each word separately, and re-joins with spaces. But `"/"` is *also* an ordinary entry in `TEXT_CODE`, mapping to the space character, because `" ": "/"` is in `MORSE_CODE`. So the simpler one-liner

```
1 "".join(TEXT_CODE[s] for s in morse.strip().split())
```

produces identical output. I confirmed this empirically: across the same test inputs above (in-

cluding the awkward leading-space, trailing-space and doubled-space cases) the naive version and the shipped version diverged on **zero** inputs.
The README's "Challenges" section defends the split-on-" / " approach with "splitting naively on / alone would have left stray spaces around each word". That reasoning is about splitting on the slash, but the alternative it overlooks is not splitting on the slash at all, and simply *decoding* it like any other symbol. Two mechanisms now encode the same rule, and they have to agree forever. This is not a bug and it does not misbehave; it is redundant complexity with a rationale in the README that does not survive contact with the code.

3.7 The command-line fallback

```
if __name__ == "__main__":
    message = input("Enter a message to be converted into Morse code: ")
    print(text_to_morse(message))
```

Running `python3 morse_converter.py` gives a one-shot prompt. Note it is **one-directional**: the CLI can encode but not decode, even though `morse_to_text` sits right above it in the same file. Note also that it does not catch `ValueError`, so typing an unsupported character here produces a raw Python traceback. Both are fine for a development convenience, and the README is upfront that this is a leftover from the project's origin as a bare script.

4 `morse_audio.py`: turning dots into sound

This is the most technically interesting file, and the one whose central claim ("real audio with no dependencies") is worth understanding properly.

4.1 The timing model

Morse code is not just a sequence of dots and dashes; it is a sequence of *durations*. The standard defines everything in terms of one base unit:

Element	Duration	Constant in the code
dot	1 unit of tone	`DOT_UNITS = 1`
dash	3 units of tone	`DASH_UNITS = 3`
gap between symbols of one letter	1 unit of silence	`INTRA_SYMBOL_GAP_UNITS = 1`
gap between letters	3 units of silence	`INTER_LETTER_GAP_UNITS = 3`
gap between words	7 units of silence	`INTER_WORD_GAP_UNITS = 7`

Table 1: The standard Morse timing ratios, each given a named constant at the top of `morse_audio.py`. One unit is set to 80 ms (`UNIT_MS = 80`), which fixes the sending speed.

Why the three different gaps matter

Silence carries information in Morse code. Without three distinct gap lengths, a listener cannot tell where one letter ends and the next begins, and `....  ..` ("HI") would be indistinguishable from a single six-dot blob. The gaps are not padding, they are punctuation.

4.2 `build_timing_sequence`: the single source of truth

This function converts a Morse string into an explicit list of `(kind, units)` pairs, where `kind` is `"dot"`, `"dash"` or `"gap"`.

build_timing_sequence(morse)

```
events := empty list
words := morse.strip() split on " / ", dropping empty pieces
for each word, with index w:
    letters := word split on " ", dropping empty pieces
    for each letter, with index l:
        symbols := the characters of letter that are "." or "-"
        for each symbol, with index s:
            emit ("dot",1) if ".", else ("dash",3)
            if s is not the last symbol:  emit ("gap",1)
        if l is not the last letter:  emit ("gap",3)
    if w is not the last word:  emit ("gap",7)
return events
```

The three "if not the last" checks are the whole trick. Gaps are emitted *between* elements, never after the final one, so a message never ends with trailing silence and gaps never double up at a boundary.

For `"SOS"` this produces 17 events totalling **27 units**. Repeating the call as `"SOS SOS"`, which is how a distress signal is actually sent, produces 35 events totalling **61 units** and adds the one thing a single word cannot show: the 7-unit word gap. Here is the real timeline it generates, drawn to scale:

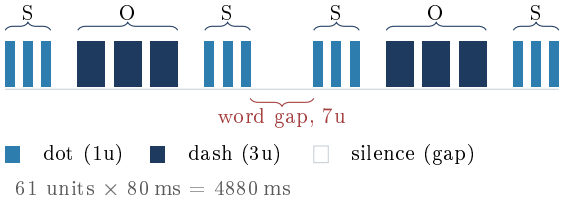

Figure 4: The real event timeline `build_timing_sequence` emits for `"SOS SOS"`, drawn to scale. Each S is three 1-unit dots and each O is three 3-unit dashes, separated by 1-unit gaps inside the letter and 3-unit gaps between letters, with the single 7-unit word gap in the middle. The dash being visibly three times the dot is the ratio the project originally got wrong. This diagram was drawn from the function's actual output, not from the specification.

One list drives both the eyes and the ears

This event list is the project's best design decision. `gui.py` calls `build_timing_sequence` **once** and uses the result for both the on-screen flashing and the tone playback. Because both read the same list, the sound and the picture cannot drift apart. The obvious alternative, hand-tuning a visual delay and an audio delay separately, is exactly what the project used to do, and the README records that it produced a dot-to-dash ratio of roughly 1:1.9 instead of the correct 1:3.

Silent tolerance of garbage, inconsistent with the decoder

The line `symbols = [c for c in letter if c in ".-"]` keeps only dots and dashes and **silently discards everything else**. Feeding `build_timing_sequence(".- XYZ -...")` does not raise; it drops `XYZ` entirely but still emits the inter-letter gap around it, producing a timeline with two adjacent 3-unit gaps and no symbol between them (I confirmed this by calling the function directly).

So the two halves of the program disagree about invalid input: `morse_to_text` raises a `ValueError` naming the bad symbol, while `build_timing_sequence` shrugs and carries on. In the GUI this is masked, because invalid Morse never reaches playback via the normal path,

but it is a real inconsistency in the library's contract and worth owning rather than explaining away.

4.3 Generating a tone from nothing

Jargon: Digital audio, PCM, and sample rate

A sound is a wave of air pressure. A computer stores it by measuring that pressure many thousands of times per second and writing down each measurement as a number. Each measurement is a *sample*; the number of measurements per second is the *sample rate* (here `SAMPLE_RATE = 44100`, or 44,100 per second, the same rate used by audio CDs). This raw list of numbers is called *PCM* (Pulse-Code Modulation). "16-bit" means each number is stored in 16 bits, giving a range of −32768 to +32767.

A pure musical tone is a *sine wave*: a value that swings smoothly up and down at a fixed rate. The number of swings per second is the *frequency*, measured in hertz (Hz), and it is what we perceive as pitch. This project uses 700 Hz (`TONE_FREQUENCY_HZ = 700`), a typical Morse practice pitch.

```
def _sine_pcm_frames(duration_ms, frequency=TONE_FREQUENCY_HZ,
                     sample_rate=SAMPLE_RATE, volume=VOLUME):
    """Raw 16-bit little-endian PCM bytes for a pure sine tone."""
    n_samples = int(sample_rate * duration_ms / 1000)
    amplitude = int(volume * 32767)
    samples = [
        int(amplitude * math.sin(2 * math.pi * frequency * (i / sample_rate)))
        for i in range(n_samples)
    ]
    return struct.pack("<%dh" % len(samples), *samples)
```

Listing 2: The entire tone generator.

This is the crux of the "no dependencies" claim, so it is worth reading closely:

- `n_samples` is how many measurements this tone needs: 80 ms at 44100 samples per second is 3528 samples.
- `amplitude` scales the wave's height. `VOLUME = 0.5` means the tone peaks at half the maximum a 16-bit sample can hold, leaving headroom.
- `math.sin(2 * math.pi * frequency * (i / sample_rate))` is the sine wave itself. `i / sample_rate` converts the sample number into a time in seconds; multiplying by the frequency and by 2π (one full cycle in radians) gives the point on the wave at that moment.
- `struct.pack("<%dh" % len(samples), *samples)` converts the Python list of numbers into raw bytes. `<` means little-endian (least significant byte first, the byte order WAV files use), and `h` means "signed 2-byte integer". The `%d` is filled in with the count, so the format string becomes e.g. `"<3528h"`.

Jargon: The `wave` module and the WAV format

A WAV file is essentially raw PCM samples with a small header on the front declaring the sample rate, how many channels there are (1 = mono, 2 = stereo), and how many bytes each sample uses. Python's standard library includes `wave`, which writes that header for you. This is why the project needs no audio library: the file format is simple enough that the standard library already covers it.

`write_tone_wav` is the thin wrapper that puts those two together: generate the frames, open a WAV file, declare mono (`setnchannels(1)`), 2 bytes per sample (`setsampwidth(2)`), the sample rate, and write.

4.4 The tone cache

```
_tmp_dir = None

def _tone_dir():
    global _tmp_dir
    if _tmp_dir is None:
        _tmp_dir = tempfile.mkdtemp(prefix="morse_tones_")
        atexit.register(shutil.rmtree, _tmp_dir, ignore_errors=True)
    return _tmp_dir
```

The pattern here is *lazy initialisation*: the temporary directory is created the first time it is needed, not at import time, and only once. `atexit.register` schedules the directory's deletion for when Python exits, so the program cleans up after itself without anyone having to remember to call a cleanup function.

`build_symbol_tones` then writes exactly two files into that directory, `dot.wav` (80 ms of tone) and `dash.wav` (240 ms of tone), once per app run. Every dot in every message replays the same file. This is a sensible trade: two small files generated once, rather than a fresh sine wave computed for every symbol.

4.5 Finding something that can actually make a sound

Generating a WAV is the easy half. *Playing* it is where portability bites, because Python's standard library has no cross-platform way to send audio to a speaker. The project's answer is to ask the machine what it has, at runtime.

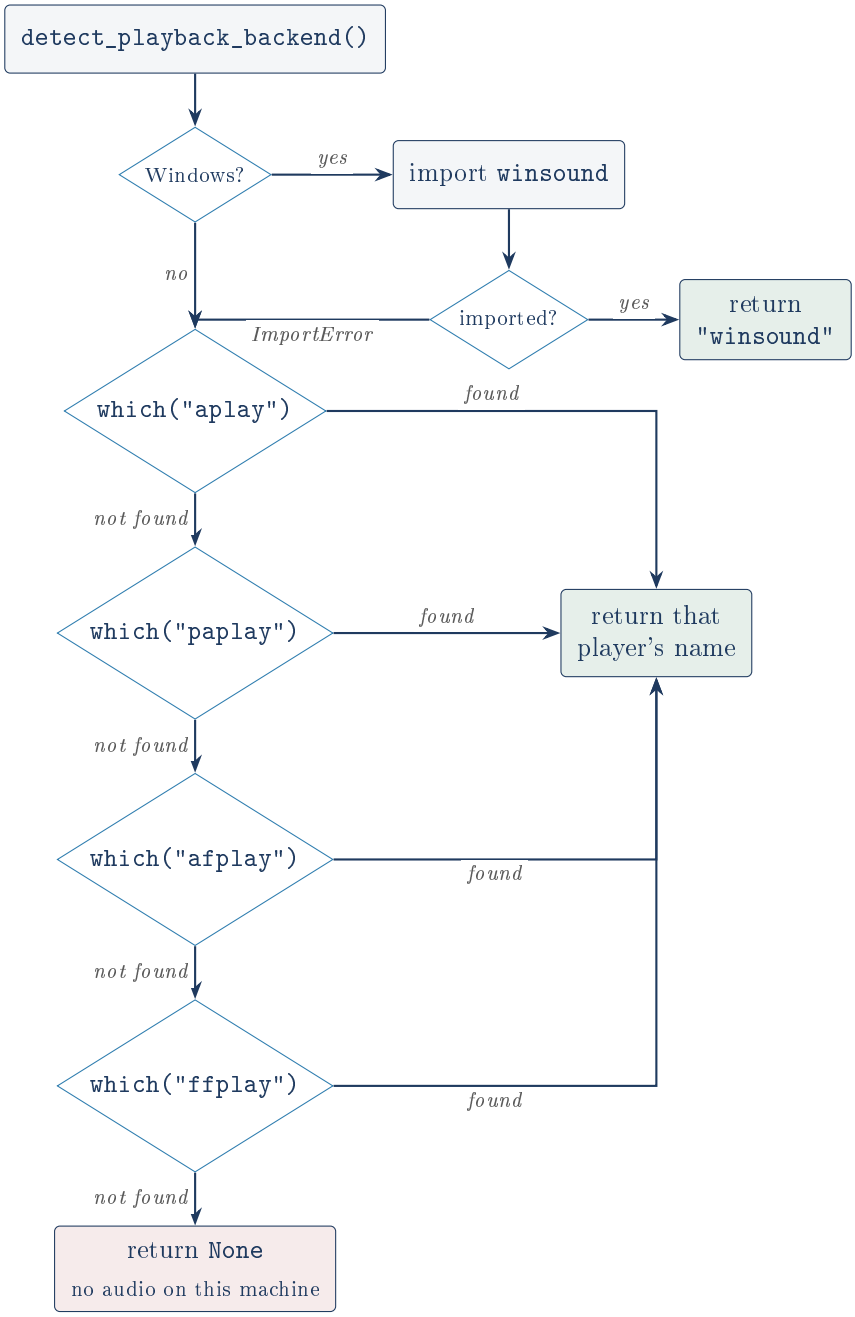

Figure 5: Backend detection. Nothing is assumed about the host: every candidate is checked for real, and `None` is a legitimate, handled answer.

Jargon: `shutil.which`

A standard-library function that answers "if I typed this command name in a terminal, would anything run, and where is it?" It searches the directories listed in the `PATH` environment variable and returns the full path, or `None`. It is the programmatic version of the Unix `which` command.

Detecting rather than assuming

The alternative approach, "I am on Linux, therefore `aplay` exists", is wrong often enough to matter: a minimal container may have neither ALSA nor PulseAudio installed. By calling `shutil.which` and treating `None` as a normal outcome, the program can tell the user the truth

("No audio output available on this system, showing visual playback only") instead of crashing or, worse, silently doing nothing and looking broken.

On this machine, I ran `detect_playback_backend()` and it returned `'aplay'`, matching what the README claims.

4.6 `play_wav_async`: fire and forget

Each backend needs a different command line, so the function maps the backend name to its arguments (`["aplay", "-q", path]`, `["ffplay", "-nodisp", "-autoexit", "-loglevel", "quiet", path]`, and so on), then launches it with `subprocess.Popen`.

Jargon: Process, `subprocess.Popen`, and blocking

A *process* is a running program. `subprocess.Popen` starts another program as a separate process and **returns immediately**, without waiting for it to finish. Its cousins `subprocess.run` and `.wait()` instead *block*: they freeze the caller until the other program exits.

Why `Popen` and not `run` is load-bearing

A GUI is a single loop constantly redrawing the window and reacting to input. If that loop blocks for 240 ms waiting for a dash's tone process to exit, the window freezes for 240 ms: the flash cannot turn off on time, and the picture falls behind the sound. Using `Popen` keeps tkinter's own timer in charge of the whole timeline, with audio fired off alongside it. The README identifies this correctly.

`stdout` and `stderr` are redirected to `DEVNULL` (a black hole) so the players cannot spray output into the terminal.

Fire and forget means never knowing

The consequences of this design are real and the README states them: each symbol pays the cost of spawning a whole operating-system process, and the program never learns whether a tone actually played. The processes are never collected either (the `Popen` object is returned and then discarded by the GUI), so on Unix each finished player leaves a *zombie* entry in the process table until the Python process exits. For a program that plays a few dozen short tones this is harmless, and the alternative (an audio device API) means a third-party dependency the project deliberately refuses. It is a trade-off, honestly made, not an oversight.

4.7 `render_morse_to_wav`: the offline renderer

This function renders an *entire* message, tones and silences alike, into one WAV file, returning the event list it used.

Unused by the application

Nothing in `gui.py` calls `render_morse_to_wav`. Its only caller is the `__main__` smoke test at the bottom of its own file. By a strict definition it is dead code in the shipped application. The README is upfront that it exists for offline rendering and was the tool used to verify timing during development, which is a legitimate reason to keep it, and it earned its place again while I was checking this project (see below). But it should be described as a development instrument, not as an application feature.

I used it to check the timing claim independently. Rendering `"SOS"` and inspecting the resulting file gives **95,256 frames at 44,100 Hz, exactly 2160 ms**, matching the 27 units × 80 ms computed by hand from the dot/dash/gap rules. Rendering `"SOS SOS"` gives **215,208 frames, exactly 4880 ms**, matching 61 units × 80 ms, so the 7-unit word gap is accounted for to the sample. The timing claim is confirmed, not merely asserted.

No fade on the tone edges (inferred, not observed)

Each tone begins and ends abruptly at whatever point of the sine wave it happens to reach; there is no short fade-in or fade-out (an *envelope*). In audio work, a hard cut mid-wave typically produces an audible click. I am flagging this as **inferred from the code, not confirmed by listening**: I read the generator and no envelope is applied, but I did not verify by ear whether a click is actually perceptible at this volume and pitch. It is not a correctness problem in any case.

5 gui.py: the window

Jargon: tkinter, widgets, and the event loop

tkinter is Python's built-in GUI toolkit, a wrapper around the older Tk library. It ships with Python (though on Linux it sometimes needs a separate OS package such as `python3-tk`). A *widget* is any interface element: a window, button, text box, canvas. *ttk* is a newer set of themed widgets that look more native. The *event loop* (`root.mainloop()`) is an endless loop that waits for things to happen (a key press, a click, a timer expiring) and calls the function you registered for that event.

Jargon: Callback

A function you hand to someone else so they can call it later, when something happens. `command=self._start_playback` on a button does not call `_start_playback`; it stores it, to be called when the button is clicked.

5.1 Construction order

`MorseApp.__init__` does four things in a deliberate order:

1. Configure the window: title, dark background, 760×560 starting size, and a `minsize` of 620×480 so the layout cannot be crushed.
2. Set `_playback_job = None`, the handle used to cancel a scheduled animation step.
3. **Detect audio before building anything visible.** The result decides what the playback panel will say, so it has to be known first. If a backend exists, the two tone files are generated now, at startup, rather than during playback when the delay would be audible.
4. Build the styles, then the layout.

5.2 Styling

`_build_style` sets up a hand-rolled dark theme: a set of named colour constants at the top of the file (`BG`, `PANEL_BG`, `ACCENT` and so on) applied through `ttk.Style`. It first tries `style.theme_use("clam")` inside a `try`/`except tk.TclError`, because `clam` is the theme that actually honours custom colours, and the `except` means the app still opens on a system where that theme is missing rather than dying at startup. This is a small and well-judged piece of defensive code.

5.3 The tab factory

Rather than writing two nearly identical tabs, `_build_conversion_tab` is called twice with different arguments and returns a dictionary describing the tab it just built:

```
tab_state = {
    "frame": frame, "input_box": input_box, "output_box": output_box,
    "error_label": error_label, "convert_fn": convert_fn,
    "playback_source": playback_source,
}
```

The key idea is `convert_fn`: the tab does not know *which* direction it converts. It just calls the function it was handed. The "Text to Morse" tab gets one function, the "Morse to Text" tab gets the other, and the identical machinery serves both.

Jargon: Passing a function as a value

In Python, functions are ordinary values: they can be stored in a dictionary and called later, exactly like a number or a string. This is what lets one piece of code serve two directions.

Two wrapper functions that wrap nothing

```
def _safe_text_to_morse(self, raw: str) -> str:
    return text_to_morse(raw)

def _safe_morse_to_text(self, raw: str) -> str:
    return morse_to_text(raw)
```

These are pure pass-throughs. They add no behaviour at all, and the `_safe_` prefix promises a safety they do not provide: neither catches anything, and the actual error handling lives in `_on_key_release`. A reader scanning the file would reasonably assume these are where input is made safe, and be wrong. They could be deleted and replaced with the imported functions directly. This is a small piece of misleading dead abstraction.

5.4 Live conversion on every keystroke

Each input box binds `<KeyRelease>` to `_on_key_release`: after every key comes back up, the whole box is re-read and re-converted. There is no submit button because there is no submit.

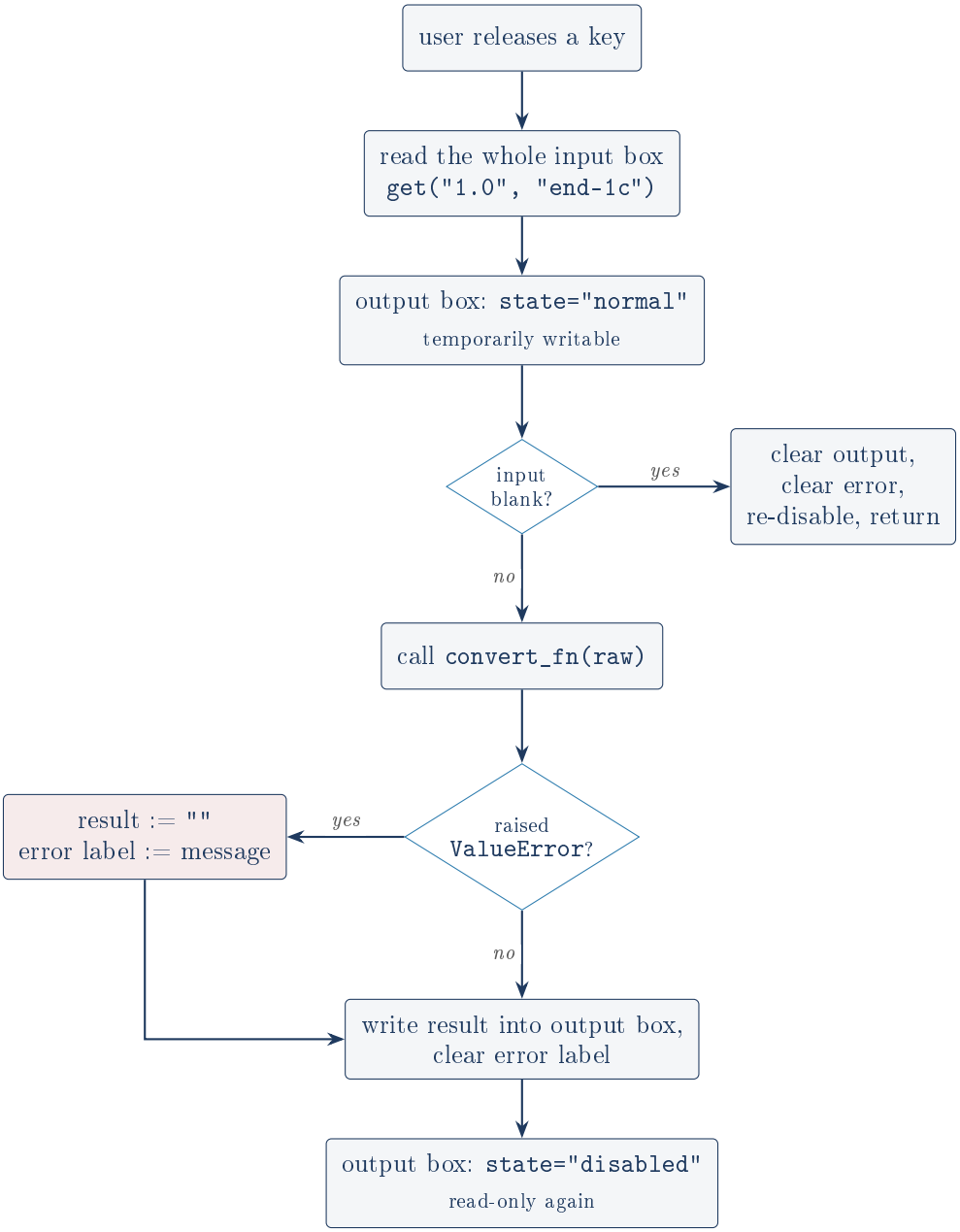

Figure 6: What happens on every single keystroke.

Two details are worth naming:

- **The disabled-output dance.** The output box is kept `state="disabled"` so the user cannot type into it. But a disabled tkinter `Text` rejects programmatic writes too, so the code must flip it to `"normal"`, write, and flip it back. That is why `state` is toggled three times in one short function.
- **`"1.0"` and `"end-1c"`.** tkinter addresses text as "line.column", 1-indexed lines and 0-indexed columns, so `"1.0"` is the very start. `"end-1c"` means "the end, minus one character", because tkinter always appends an invisible trailing newline that you almost never want.
- **Catching `ValueError` is the whole point.** Because conversion runs mid-word, invalid intermediate states are normal, not exceptional. The `except ValueError` turns what would be a crash into a small inline message.

A multi-line box that cannot accept multi-line input

Both input boxes are `tk.Text` widgets with `height=6`: they look and behave like multi-line editors, and pressing Enter inserts a newline. But `"\n"` is not in `MORSE_CODE`, so the moment a user presses Enter the conversion fails with `No Morse mapping for character: '\n'` and the output box goes blank. I confirmed this by calling `text_to_morse("A\n")` directly: it raises. Tab characters behave the same way.
The failure is handled gracefully (an inline message, not a crash), so this is a usability wrinkle rather than a defect: the widget invites input the alphabet cannot represent. Mapping newline to a word separator, or stripping it before conversion, would close the gap.

5.5 The playback engine

Playback is the most intricate part of the GUI, and it is built as a small state machine driven by tkinter's timer.

Jargon: `root.after(ms, fn)` and cooperative animation

`after` asks tkinter to call `fn` once, roughly `ms` milliseconds later, and returns a job handle that `after_cancel` can use to call it off. Nothing sleeps and nothing blocks: the event loop keeps running and simply fires the callback when the time comes. Chaining `after` calls (each callback scheduling the next) is the standard way to animate in tkinter without freezing the window.

`_start_playback` runs when Play is clicked:

1. Cancel any animation already in flight (`after_cancel`), so a second click restarts cleanly rather than running two animations at once.
2. Ask `_current_tab_state()` which Morse string to play.
3. Clear the canvas; if there is no Morse, stop here.
4. Call `build_timing_sequence(morse)` to get the event list.
5. `_render_events` lays out one shape per symbol on the canvas, dark (`SYMBOL_OFF`), and pairs each with its real millisecond duration.
6. Set `_play_index = 0` and call `_advance_playback`.

`_render_events` draws a small oval for each dot (14 pixels wide) and a longer rectangle for each dash (34 pixels), so the shapes themselves encode the distinction. Gaps get no shape, but they do advance the horizontal cursor by `6 + units * 4` pixels, so a word gap opens a visibly wider space than a letter gap. The visual spacing is proportional to the real timing but not on the same scale as the shapes, which is a presentational choice, not a timing one.

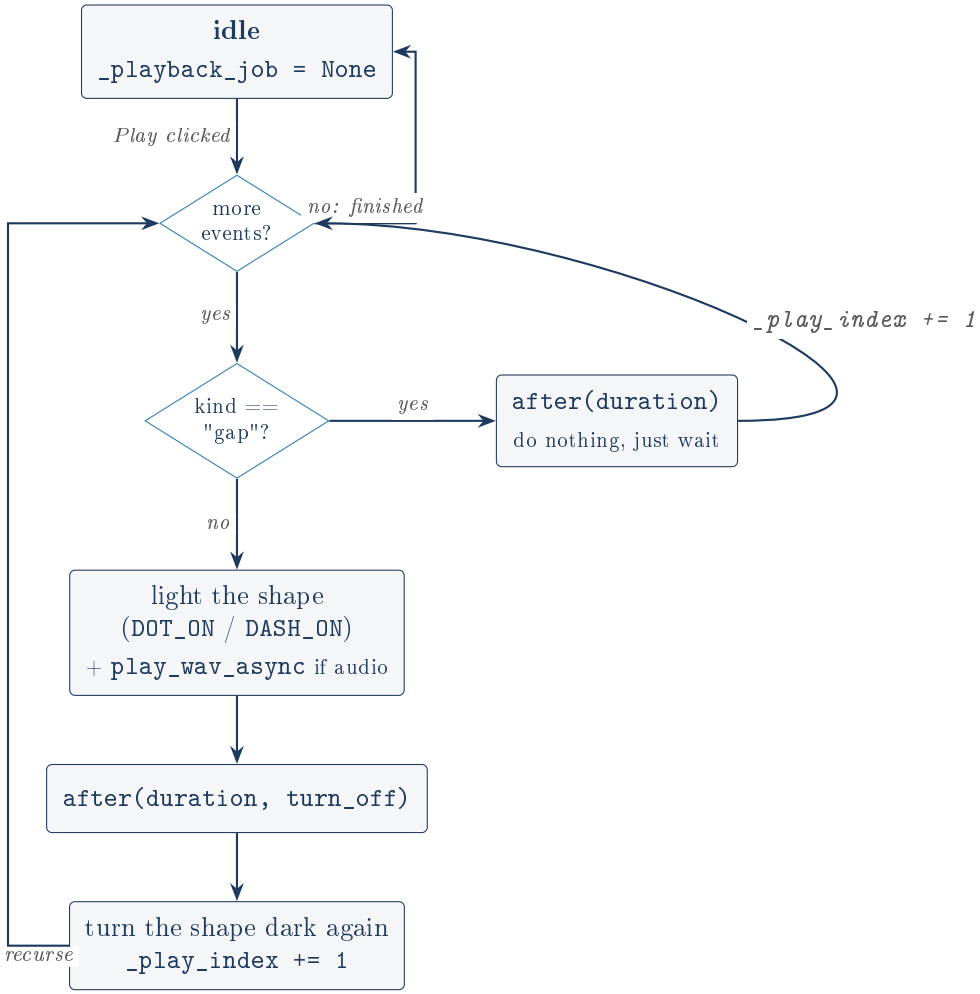

Figure 7: The playback state machine in `_advance_playback`. Each step schedules the next one through `after`; the loop is a chain of timer callbacks, never a blocking loop.

For a dot or dash, the sequence within one step is: light the shape, fire the tone asynchronously, and schedule `turn_off` for `duration_ms` later. `turn_off` darkens the shape, increments the index, and calls `_advance_playback` again. The audio and the light therefore start on the same line of code and are extinguished by the same timer.

The `try`/`except OSError` around `play_wav_async` is deliberate: if spawning the player fails partway through a message, the visual playback continues uninterrupted. The code comment states the reasoning plainly, that the flash is an honest representation on its own.

`_current_tab_state` is misnamed and does not consider the current tab

Despite the name, this function returns **a string**, not a tab state dictionary, and it never asks the notebook which tab is actually selected. It checks the Text-to-Morse tab's output box first and only falls back to the Morse-to-Text tab's input box if the first is empty.
The consequence: type text in the first tab, switch to the second, type Morse there, and press Play. It plays the *first* tab's content, because that box is still populated. The README's final "Challenges" bullet acknowledges the fixed priority honestly and explains that the architecture diagram was redrawn to reflect it, which is to the author's credit. But the priority itself is still surprising behaviour, and the function name actively misdirects a reader about both its return type and its logic.

Attributes created outside `__init__`

`_playback_events` and `_play_index` are first assigned inside `_start_playback`, not in `__init__`. If `_advance_playback` were ever called before a first Play click, it would fail with `AttributeError`. No such path exists today (only `_start_playback` begins the chain), so this is a latent fragility rather than a live bug, and initialising both in `__init__` would cost one line.

5.6 The clipboard

`_copy_to_clipboard` uses `clipboard_clear()` and `clipboard_append()`, both built into tkinter. This is a genuinely nice detail: copying to the system clipboard is the classic reason a small Python project reaches for a third-party package, and it turns out not to be necessary.

A known tkinter clipboard caveat (inferred, not tested here)

On X11-based Linux systems, clipboard contents are owned by the application that set them, and are conventionally lost when that application exits unless a clipboard manager is running. So text copied from this app may not survive closing the window. I am flagging this as **inferred from how X11 clipboards are documented to work, not verified against this application**, and it affects nothing while the app is open.

6 End to end: one keystroke and one click

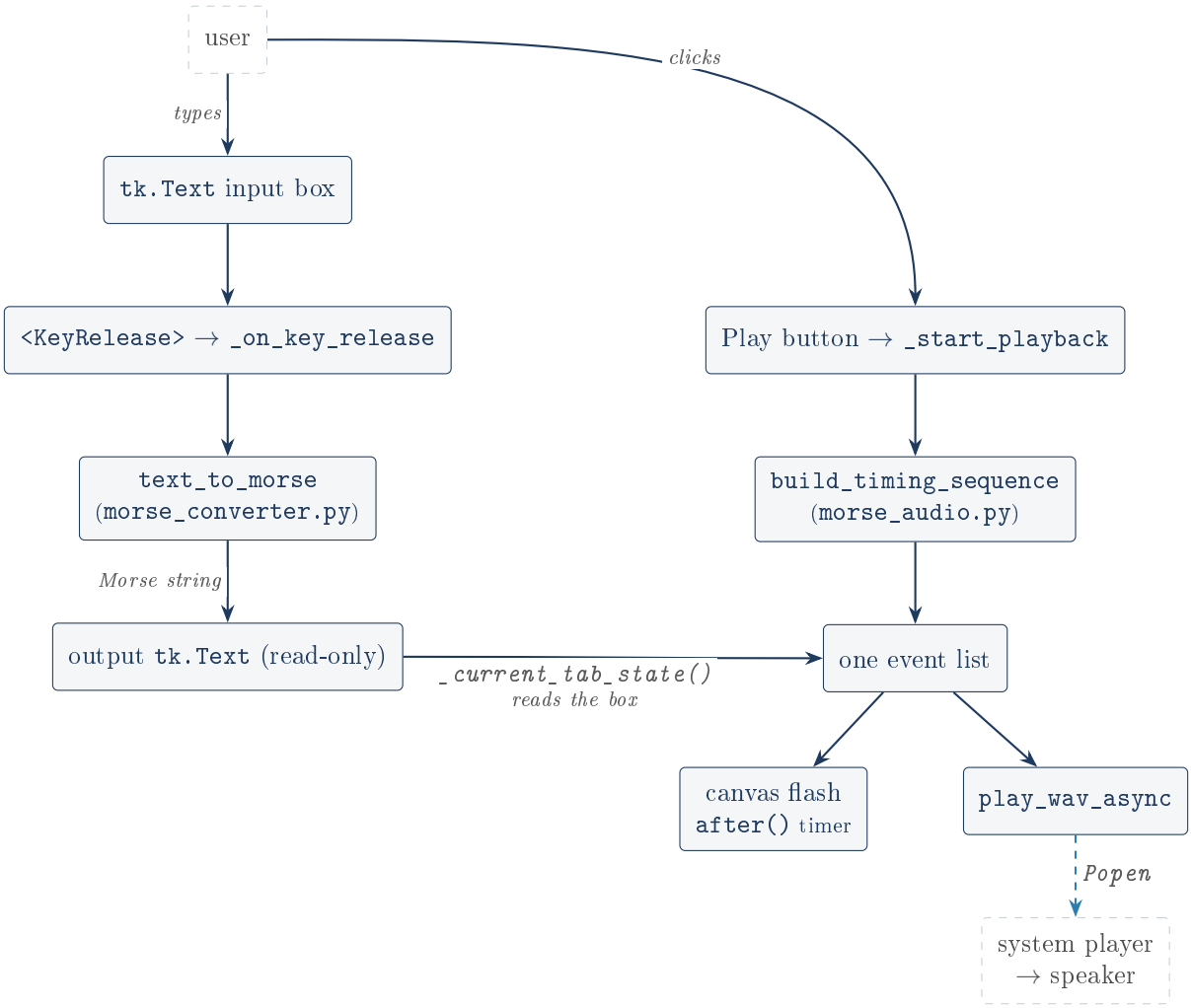

Figure 8: The complete data flow. The important feature is the fork at the bottom: one event list feeds both the eyes and the ears, so they cannot disagree.

7 Dependencies, configuration and licence

`requirements.txt` contains one line, a comment stating there are no third-party dependencies. The file exists so the answer to "what does this need installed?" is explicit rather than absent.

Every import in the project is standard library:

Module	Used in	Role
`tkinter`, `tkinter.ttk`	`gui.py`	the entire interface
`math`	`morse_audio.py`	`sin` and `pi` for the sine wave
`struct`	`morse_audio.py`	pack sample numbers into raw bytes
`wave`	`morse_audio.py`	write the WAV header and frames
`shutil`	`morse_audio.py`	`which` to find a player; `rmtree` to clean up
`subprocess`	`morse_audio.py`	launch the player without blocking
`tempfile`	`morse_audio.py`	a scratch directory for the two tones
`atexit`	`morse_audio.py`	delete that directory on exit
`os`	`morse_audio.py`	join paths portably
`sys`	`morse_audio.py`	`sys.platform` to detect Windows

Table 2: Every dependency, and why it is there. `morse_converter.py` imports nothing at all.

There is no configuration file, no environment variable, no network access, and no persistence: the program writes only two temporary WAV files, and deletes them on exit. The tuning knobs (`UNIT_MS`, `TONE_FREQUENCY_HZ`, `VOLUME`, `SAMPLE_RATE`, and the colour constants) are module-level constants, editable only by editing the source. For a project this size that is a reasonable stopping point, not a gap.

`.gitignore` excludes the usual Python build and cache artefacts (`__pycache__/`, `*.pyc`, virtual environments, coverage output) plus `.env`, ensuring a credentials file could never be committed by accident. The licence is PolyForm Strict 1.0.0: source-available for viewing, with no permission to use it in a product, which is the appropriate choice for a portfolio piece.

8 What I verified by running it

Nothing in this document about behaviour is taken on trust from the README. The following were executed against the code in this folder:

- **Table integrity.** `MORSE_CODE` has 57 entries; `TEXT_CODE` has 57; zero Morse patterns are duplicated, so the inversion loses nothing.
- **Round trip.** `morse_to_text(text_to_morse(s)) == s.upper()` held for every input tested, including mixed case, punctuation, digits, a literal `/` in the text, leading/trailing spaces, doubled internal spaces, and an all-whitespace string.
- **The redundancy claim.** A naive one-line decoder that never splits on `" / "` produced identical output to `morse_to_text` on every test input, confirming the outer split adds no behaviour.
- **Timing.** `build_timing_sequence` for `"SOS"` (`... --- ...`) emits 17 events totalling 27 units, and for `"SOS SOS"` 35 events totalling 61 units. `render_morse_to_wav` produced files of 95,256 and 215,208 frames at 44,100 Hz, exactly 2160 ms and 4880 ms, matching 27×80 ms and 61×80 ms computed by hand from the standard ratios. The timing claim is true.
- **Backend detection.** `detect_playback_backend()` returned `'aplay'` on this machine, matching the README.
- **The newline limitation.** `text_to_morse("A\n")` raises `ValueError`, as does a tab, as does an accented letter.
- **Silent garbage tolerance.** `build_timing_sequence(".- XYZ -...")` returns a timeline with `XYZ` dropped and no error raised.

I did not run the GUI itself (this session has no display), so statements about on-screen behaviour are read from the code rather than observed. The project includes a screenshot at

`docs/screenshots/app.png` showing the running window. I also could not test the `winsound`, `afplay` or `paplay` paths, which need Windows, macOS, or PulseAudio respectively; the README already states this limitation itself.

9 Honest summary

What is genuinely good here:

- The separation of logic, audio and interface is real, not nominal, and it is the reason the audio module could be rewritten without touching the converter.
- The single event list feeding both the visual and the audio timeline is the right answer to a problem the project got wrong the first time, and the README documents that history instead of hiding it.
- Runtime backend detection with a genuine no-audio fallback is more care than a practice project usually gets.
- Generating real sound from `math.sin`, `struct` and `wave` is a neat, correct, and well-earned way to keep the dependency list empty.
- The verified timing holds to the millisecond.

What is weak, in order of how much it matters:

1. **No automated tests.** For a project whose correctness lives in a 57-entry table, the absence of a test file is the biggest gap. Every claim in this document had to be re-established by hand. The README names this itself as the first thing it would do differently, which is the right instinct.
2. `_current_tab_state` **is misnamed and ignores the selected tab**, producing genuinely surprising Play behaviour when both tabs have content.
3. `build_timing_sequence` **silently drops invalid symbols** while `morse_to_text` raises on them: two different contracts for the same malformed input.
4. **The redundant** `" / "` **split**, defended in the README by an argument that does not address the simpler alternative.
5. `_safe_text_to_morse` **and** `_safe_morse_to_text` **are empty wrappers** whose names promise safety they do not deliver.
6. **Multi-line input boxes that reject newlines**, handled gracefully but still a mismatch between the widget and the alphabet.
7. `render_morse_to_wav` **is unused by the application**, and the unreaped `Popen` processes leave short-lived zombies.

None of these is a crash, a security issue, or a wrong answer. They are the ordinary residue of a project that grew from a script into an application, and the README is unusually candid about most of them already. The gap between the code and its documentation is small, and where it exists (the `" / "` rationale) it is a matter of a defence that no longer fits rather than a false claim about what the software does.